Flutter and Dart Programming

Flutter is a framework that makes it easier to build apps with a beautiful user interface (UI) that works on both Android and iOS devices. **Dart** is the programming language behind Flutter. It's simple, fast, and designed to be easy to learn, especially for beginners.

Basic Concepts in Dart Programming

1. Dart variable
   - To store information like number or text.
   - To declare a variable, use types like var, int, double, String, etc.

```
var name = 'Alice';  // A string variable
int age = 30;        // An integer variable
```

2. Dart operators
   - Operators are used to perform operations on variables and values, like adding, subtracting, comparing, or assigning values.
   - For more operators information and how to use it, you can check here.

```
int sum = 5 + 3;     // Addition
bool isEqual = (5 == 5);  // Comparison
```

3. Dart keywords
   - Keywords are special words that have predefined meanings in Dart, such as if, for, class, void, etc. These words cannot be used as variable names.
   - For more keywords information, you can check here.
4. Dart Build in types (type data)
   - Dart supports several built-in types that allow you to work with different kinds of data:
     - ◊ Numbers (int, double)
       int for whole numbers, and double for decimal numbers.

```
int x = 10;
double y = 3.14;
```

     - ◊ Strings (String)
       Used to represent text.

```
String greeting = 'Hello, World!';
```

     - ◊ Booleans (bool)
       Represent `true` or `false`.

```
bool isActive = true;
```

     - ◊ **Records** ((value1, value2))
       A simple way to group multiple values.

```
var record = (10, 'hello');
```

◊ **Functions** (Function)
Functions perform actions and can return a results.

```
int add(int a, int b) {
   return a + b;
}
```

◊ **Lists** (List, also known as *arrays*)
Used to store multiple values in an ordered sequence.

```
List<String> colors = ['red', 'blue', 'green'];
```

- **.join()** method to combines all list elements into a single string. Can specify a custom separator (e.g., comma, space, hyphen) between elements.

```
List<String> fruits = ['Apple', 'Banana', 'Cherry'];
print(fruits.join(', ')); // Output: "Apple, Banana,
Cherry"
```

◊ **Sets** (Set)
A collection of unique items.

```
Set<int> uniqueNumbers = {1, 2, 3};
```

◊ **Maps** (Map)
A collection of key-value pairs.

```
Map<String, int> scores = {'Alice': 90, 'Bob': 85};
```

5. Dart Generic
   - Generics allow to write flexible code that works with different data types.
   - For example, you can create a list that holds only `String` values or only `int` values by using generics.

```
List<int> numbers = [1, 2, 3];
```

6. Dart Error Handling
   - Error handling helps manage and respond to problems that occur during the execution of your program.
   - In Dart, you use `try`, `catch`, and `finally` to handle errors.

```
try {
  int result = 10 ~/ 0;
} catch (e) {
  print('Cannot divide by zero: $e');
}
```

7. Dart OOP
   - Dart supports object-oriented programming, where you can create classes to represent objects.
   - These objects can have properties (variables) and methods (functions).

```
class Car {
  String make;
```

```
  String model;

  Car(this.make, this.model);

  void displayInfo() {
    print('Car: $make $model');
  }
}

void main() {
  var myCar = Car('Toyota', 'Corolla');
  myCar.displayInfo();
}
```

◊ Enum

An **enum** (short for "enumeration") is a special type in Dart that represents a fixed number of constant values.

Enums are helpful when you want to limit a variable to only accept specific values.

This makes your code safer by preventing invalid values from being assigned.

```
enum CarType { sedan, suv, truck }

void main() {
  CarType myCarType = CarType.sedan;

  if (myCarType == CarType.sedan) {
    print('Your car is a sedan.');
  }
}
```

8. Dart Null Safety
   - Null safety helps prevent errors caused by variables that are unexpectedly `null`.
   - In Dart, it is necessary to declare whether a variable can be null or not.

```
String? nullableString;  // This can be null
String nonNullableString = 'Hello';  // This cannot be
null
```

- When you need a consistent top bar for your app.
- When you want to add a title or navigation buttons.
- When you need search, settings, or more options icons.
- When you want to add tabs or extra content below the AppBar.

Text Widget

a. What is the Text Widget?
- Displays text on the screen in a Flutter app.
- Supports both plain and styled text.
- Can be used for static messages, dynamic updates, and responsive design.

b. Key Properties of the Text Widget
- Commonly used properties:

Property	Description	Example
data	The text to display (required).	`'Hello, Flutter!'`
style	Customizes the text appearance.	`TextStyle(fontSize: 18, color: Colors.blue)`
textAlign	Aligns text (`left, center, right`).	`TextAlign.center`
maxLines	Limits the number of text lines.	`maxLines: 2`
overflow	Handles text overflow (`ellipsis, fade`).	`TextOverflow.ellipsis`
softWarp	Controls text wrapping.	`softWrap: true`

c. Example: Styling Text in Flutter
- Code Example:

```
Text(
  'Welcome to Flutter!', // The text to display
  style: TextStyle(
    fontSize: 20,                  // Sets the font size
    color: Colors.green,           // Text color
    fontWeight: FontWeight.bold,   // Makes the text bold
  ),
  textAlign: TextAlign.center,      // Centers the text
  maxLines: 2,                      // Limits to 2 lines
  overflow: TextOverflow.ellipsis,  // Adds "..." if the
text is too long
)
```

- Explanation :
  - 'Welcome to Flutter!' → The text to display.
  - style → Defines size, color, weight, etc.
  - textAlign: TextAlign.center → Centers the text.
  - maxLines: 2 → Limits text to 2 lines.
  - overflow: TextOverflow.ellipsis → Shows "…" if text is too long.

d. Why Use the Text Widget?
- Advantages of using the Text widget:
  - Easy to use for both static & dynamic text.
  - Highly customizable (alignment, wrapping, overflow, etc.).

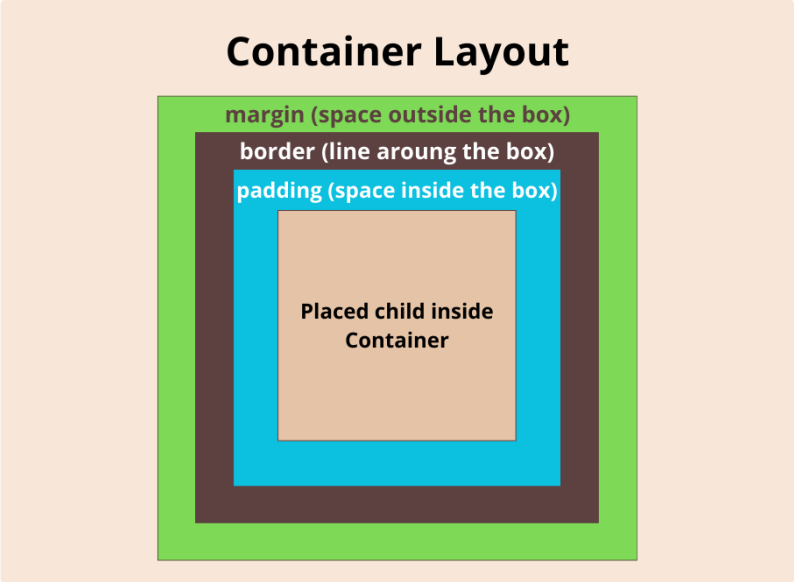
- Flexible for any layout or UI design.

Container Widget

a. What is the Container Widget?
   - A customizable "box" used to style, position, and layout elements in an app.
   - Often used to wrap other widgets and apply spacing, colors, or decorations.
   - One of the most flexible and commonly used widgets in Flutter.

b. Key Features of Container
   - Commonly used properties:

Property	Description	Example
margin	Adds space outside the container.	`margin: EdgeInsets.all(10)`
padding	Adds space inside the container.	`padding: EdgeInsets.all(15)`
width & height	Sets the size of the container.	`width: 200, height: 100`
color	Sets the background color.	`color: Colors.blue`
decoration	Adds styling like borders, gradients, and shadows.	`decoration: BoxDecoration(...)`



c. Customizing Container with BoxDecoration
   - Additional styling options with BoxDecoration:
     - color → Sets the background color.
     - border → Adds a border around the container.
     - borderRadius → Rounds the corners of the container.
     - boxShadow → Adds a shadow effect.
     - gradient → Creates a gradient background.

d. Example: Styled Container in Flutter
   - Code Example:

```
Container(
  width: 200,                // Width of the container
```